

UTN FRBA - Algoritmos y Estructuras de datos Trabajo práctico grupal

PROFESOR: ING.PABLO DAMIAN MENDEZ

COMISIÓN: K1051

INTEGRANTES		**GitHub**
Steckler, Lucas Santino	lsteckler@frba.utn.edu.ar	lstecklerutn

INTRODUCCIÓN Y OBJETIVO

El presente trabajo práctico tiene como objetivo desarrollar un programa en C + + que permita administrar una ruta de operaciones de un dron cuando no se dispone de posicionamiento GPS. Para representar el espacio de trabajo se utiliza una matriz de 200 × 200 posiciones, donde cada celda puede contener una orden asociada al recorrido del dron.

El sistema desarrollado permite cargar una ruta desde un archivo, visualizar las órdenes almacenadas en memoria, crear nuevos archivos de ataque, modificar registros tanto en un archivo como directamente en memoria y guardar nuevamente la información. Además, se incorporó una representación gráfica mediante un archivo HTML que permite visualizar la matriz y las posiciones utilizadas durante el recorrido.

A lo largo del desarrollo también se implementaron distintas validaciones con el objetivo de garantizar que las rutas cargadas sean consistentes y puedan recorrer correctamente desde una posición de despegue hasta una posición final de aterrizaje o ataque kamikaze.

REPRESENTACIÓN DE LOS DATOS

Para representar las órdenes del dron se utilizaron dos estructuras principales: **Orden** y **OrdenArchivo**.

La estructura Orden representa la información almacenada en cada posición de la matriz principal del programa. Cada orden puede indicar un tiempo de espera, el lanzamiento de una de las dos granadas, un ataque kamikaze, un aterrizaje, un despegue y las coordenadas de la siguiente posición del recorrido.

La matriz principal se definió con un tamaño de 200 × 200 posiciones:

Orden memoria[200][200]

De esta forma, cada posición de la matriz representa directamente una coordenada del espacio de trabajo. Por ejemplo, la posición memoria[5][7] corresponde a la coordenada (5,7).

Para los archivos se utilizó la estructura OrdenArchivo. Esta contiene los mismos datos que Orden, pero agrega los campos x e y. Esto es necesario

porque, al guardar una orden en un archivo, también se debe conocer en qué posición de la matriz debe ubicarse cuando el archivo sea cargado nuevamente.

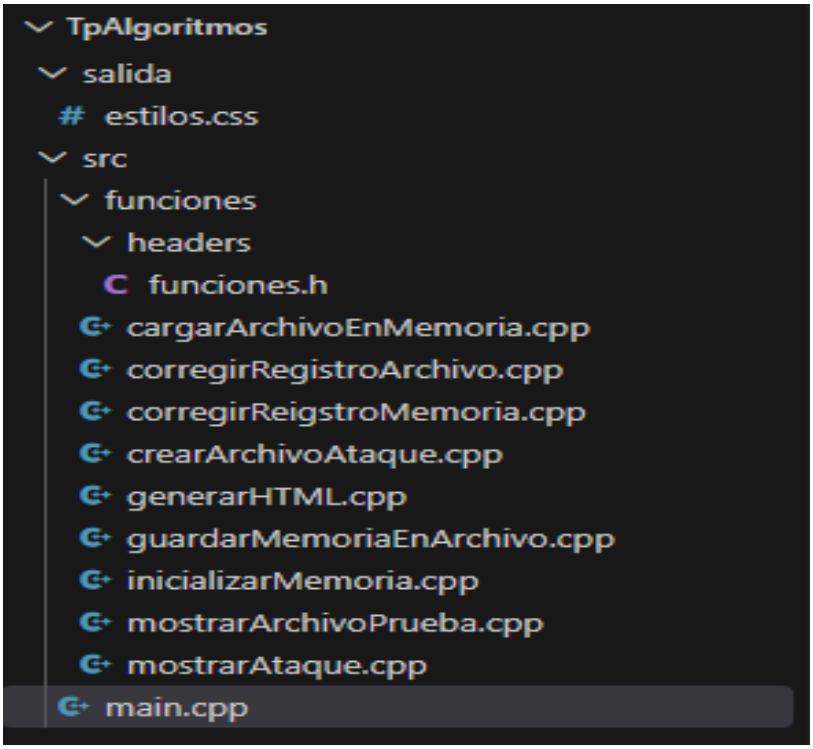
Además, se utilizó una matriz auxiliar de tipo booleano:

bool ocupada[200][200]

Esta matriz permite identificar si en una determinada coordenada existe realmente una orden cargada. Cuando ocupada[x][y] vale true, significa que esa posición contiene un registro válido. Si vale false, la posición se considera vacía.

Esta matriz auxiliar resultó útil porque una estructura Orden puede tener todos sus campos en cero o en false, por lo que solamente observando sus valores no siempre es posible determinar si la celda contiene una orden real o si nunca fue utilizada.

ORGANIZACIÓN Y MODULARIZACIÓN DEL CÓDIGO



Con el objetivo de mantener el programa ordenado y facilitar su lectura y mantenimiento, se decidió dividir el proyecto en distintos archivos y carpetas según la responsabilidad de cada parte.

La función principal se encuentra en main.cpp, donde se realiza la inicialización de las estructuras utilizadas por el programa y se presenta el menú de opciones. A partir de cada opción seleccionada, el programa deriva la ejecución hacia una función específica.

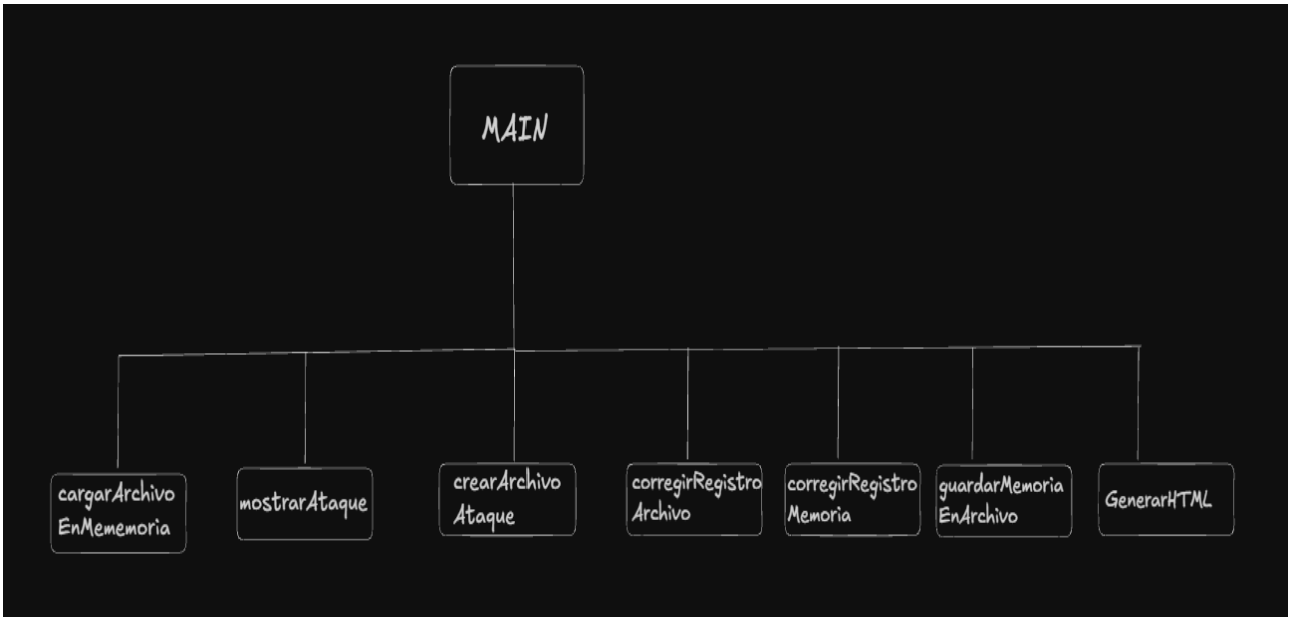
Cada funcionalidad principal fue implementada en un archivo .cpp independiente. De esta manera, tareas como cargar un archivo en memoria, mostrar el ataque, crear nuevos archivos, modificar registros, guardar la memoria o generar la visualización HTML se encuentran separadas entre sí.

Las declaraciones comunes, como las estructuras Orden y OrdenArchivo, las constantes utilizadas para definir el tamaño de la matriz y los prototipos de las funciones, fueron agrupadas en el archivo de cabecera funciones.h.

Además, se utilizó una estructura de carpetas para separar el código fuente de los archivos utilizados y generados por el programa. La carpeta src contiene el código principal, mientras que dentro de src/funciones se encuentran las distintas funciones y, dentro de headers, el archivo de cabecera.

Esta organización permite trabajar sobre cada parte del sistema de manera independiente, evita concentrar toda la lógica en un único archivo y facilita la realización de modificaciones, pruebas y correcciones durante el desarrollo.

ESTRUCTURA GENERAL DEL PROGRAMA



El programa fue dividido en distintas funciones, de manera que cada opción del menú principal tenga una responsabilidad específica. Esto permite mantener el código más ordenado y facilita tanto su lectura como su prueba.

Al comenzar la ejecución se crean la matriz principal memoria y la matriz auxiliar ocupada. Luego se inicializan todas sus posiciones y se muestra un menú que permite seleccionar una de las operaciones disponibles.

Las principales funciones utilizadas son:

- cargarArchivoEnMemoria(): carga y valida un archivo de ataque antes de copiar sus registros a memoria.
- mostrarAtaque(): muestra por pantalla las órdenes actualmente cargadas, utilizando paginación cuando es necesario.
- crearArchivoAtaque(): permite crear un nuevo archivo binario ingresando sus registros.
- corregirRegistroArchivo(): busca una orden dentro de un archivo y permite reemplazar sus datos.
- corregirRegistroMemoria(): permite modificar directamente una orden cargada en la matriz.
- guardarMemoriaEnArchivo(): guarda en un nuevo archivo las órdenes presentes en memoria.
- generarHTML(): genera una representación visual de la matriz mediante un archivo HTML.

El menú se encuentra dentro de una estructura repetitiva, por lo que vuelve a mostrarse después de ejecutar cada opción hasta que el usuario selecciona la opción de salida.

DESCRIPCIÓN DE LAS OPERACIONES DEL PROGRAMA

Cargar archivo de ataque en memoria:

La primera opción permite cargar en memoria un archivo binario que contiene las órdenes correspondientes a una misión.

El usuario ingresa el nombre del archivo que desea cargar. El programa abre el archivo en modo lectura y recorre sus registros, utilizando las coordenadas x e y

de cada OrdenArchivo para determinar la posición correspondiente dentro de la matriz.

Antes de modificar la memoria principal, los registros son almacenados en una matriz temporal. Durante este proceso se realizan las distintas validaciones requeridas para comprobar que el ataque sea válido. Entre ellas se verifica la existencia de un único despegue, un único final, que las coordenadas sean válidas y no se repitan y que el recorrido pueda completarse correctamente sin huecos ni repeticiones.

Si alguna validación falla, se informa el error y el archivo no es cargado. Si todas las verificaciones son correctas, la información temporal se copia a la matriz principal y las posiciones correspondientes se marcan como ocupadas.

Mostrar ataque cargado:

Esta opción permite visualizar por consola las órdenes que se encuentran actualmente cargadas en memoria.

La función recorre la matriz de 200 × 200 posiciones, pero únicamente muestra aquellas coordenadas que se encuentran marcadas como ocupadas. Para cada una se muestran sus acciones, el tiempo de espera y las coordenadas de la siguiente posición.

Debido a que un ataque puede contener una cantidad importante de órdenes, se implementó un sistema de paginación. Cada diez registros mostrados el programa realiza una pausa y espera que el usuario presione Enter antes de continuar con la página siguiente.

Si no existe ningún ataque cargado en memoria, el programa informa esta situación al usuario.

Crear un archivo de ataque nuevo:

Esta operación permite crear manualmente un nuevo archivo binario de ataque.

El usuario indica el nombre del archivo y luego ingresa los datos de cada orden: coordenadas, tiempo de espera, acciones y siguiente posición. Cada registro ingresado se escribe en el archivo utilizando la estructura OrdenArchivo.

Una vez almacenada una orden, el programa consulta si se desea ingresar otra. El proceso continúa hasta que el usuario indica que terminó la carga, momento en el cual se cierra el archivo.

Corregir un registro de un archivo:

Esta opción permite modificar un registro existente directamente dentro de un archivo binario.

El usuario indica el archivo y las coordenadas x e y del registro que desea modificar. El programa recorre el archivo hasta encontrar una orden que coincida con esas coordenadas.

Cuando el registro es encontrado, sus datos actuales se muestran en pantalla y el usuario puede ingresar nuevamente todos sus valores.

Para reemplazar el registro sin volver a crear el archivo completo, se utiliza fseek() para posicionarse nuevamente en el lugar correspondiente y luego fwrite() para sobreescribirlo.

Si no se encuentra ninguna orden con las coordenadas ingresadas, el programa informa que el registro no existe.

Corregir un registro en memoria:

Esta operación permite modificar una orden directamente sobre la matriz que se encuentra cargada en memoria.

El usuario ingresa las coordenadas x e y de la posición que desea modificar. Primero se verifica que las coordenadas pertenezcan a la matriz y que exista realmente una orden en esa posición mediante la matriz auxiliar ocupada.

Si la posición contiene una orden, se muestran sus valores actuales y posteriormente se permite ingresar nuevamente sus datos.

A diferencia de la operación anterior, en este caso no se modifica directamente ningún archivo, sino únicamente la información almacenada durante la ejecución del programa.

Guardar memoria en un archivo nuevo:

Esta opción permite guardar el ataque que se encuentra actualmente cargado en memoria dentro de un nuevo archivo binario.

Antes de comenzar, el programa comprueba que exista al menos una orden cargada. Si la memoria se encuentra vacía, la operación se cancela.

En caso contrario, el usuario ingresa el nombre del nuevo archivo. Luego se recorren las 200 × 200 posiciones de la matriz y solamente se escriben aquellas que están marcadas como ocupadas.

Para cada posición se crea un registro OrdenArchivo, al cual se le asignan las coordenadas correspondientes y los datos almacenados en la estructura Orden. Luego el registro se escribe mediante fwrite().

Visualizar el ataque en HTML:

La última operación genera una representación visual de la matriz de ataques mediante un archivo HTML.

El programa recorre la matriz completa de 200 × 200 posiciones y genera una tabla HTML utilizando fprintf(). Cada celda representa una coordenada de la matriz y se utiliza la información almacenada en memoria para indicar las acciones existentes en esa posición.

Para facilitar la interpretación de la tabla se incorporaron números de fila y columna, una leyenda para identificar las distintas acciones y un archivo CSS separado que permite asignar diferentes estilos visuales.

Debido al tamaño de la matriz, la visualización se encuentra dentro de un área con desplazamiento horizontal y vertical. De esta manera se conserva la tabla completa de 200 × 200 sin necesidad de mostrar las 40.000 posiciones simultáneamente en pantalla.

VALIDACIONES DEL ARCHIVO DE ATAQUE

La validación del archivo de ataque constituye una de las partes principales del programa, ya que antes de cargar una misión en memoria es necesario comprobar que sus registros forman un recorrido válido. La consigna establece que un archivo incorrecto debe ser rechazado y no debe modificar la información que ya se encuentra cargada.

Para evitar que un archivo inválido sobreescriba una misión válida que pudiera encontrarse previamente en memoria, los registros se cargan inicialmente en una matriz auxiliar denominada memoriaTemporal. Las validaciones se realizan sobre esta estructura y solamente cuando todas son superadas correctamente la información es copiada a la matriz principal.

Validación de las coordenadas:

En primer lugar se comprueba que las coordenadas x e y de cada registro se encuentren dentro de los límites de la matriz.

Como la memoria está formada por 200 filas y 200 columnas, solamente se consideran válidos valores comprendidos entre 0 y 199. Si se encuentra una coordenada fuera de ese rango, el archivo es rechazado.

También se verifica que una misma coordenada no aparezca más de una vez dentro del archivo. Para realizar este control se utiliza la matriz coordenadasUsadas. Si una posición ya se encontraba marcada y aparece nuevamente otro registro con las mismas coordenadas, se considera que el archivo contiene una coordenada repetida.

Validación del despegue:

El recorrido debe contener exactamente una instrucción de despegue. Durante la lectura del archivo se contabiliza la cantidad de órdenes que tienen el campo despegue activado.

Si no existe ninguna orden de despegue o existe más de una, el archivo no puede ser cargado. Además, se almacenan las coordenadas de la orden de despegue, ya que estas serán utilizadas posteriormente como punto de partida para recorrer la misión.

También se comprueba que la instrucción de despegue no se encuentre acompañada por una acción de ataque. Esta es una de las restricciones establecidas específicamente por la consigna.

Validación de la finalización de la misión:

La misión debe poseer un único final. Este puede producirse mediante una orden de aterrizaje o mediante un ataque kamikaze.

Para verificar esta condición se contabilizan ambas acciones y se comprueba que la suma de aterrizajes y ataques kamikaze sea exactamente igual a uno.

De esta manera se rechazan tanto las rutas que no poseen ningún final como aquellas que contienen más de una forma de finalización. La consigna establece que debe existir un único aterrizaje o un ataque kamikaze que finalice la operación.

Validación del recorrido:

Una vez realizadas las verificaciones generales, el programa comienza a recorrer la misión partiendo desde la coordenada donde se encuentra el despegue.

El orden físico de los registros dentro del archivo no determina el recorrido. La ruta se obtiene utilizando los campos siguientex y siguientey de cada orden, que indican cuál es la próxima posición que debe visitar el dron.

Para controlar las posiciones que ya fueron recorridas se utiliza otra matriz booleana denominada recorrido. Cada vez que se visita una posición, esta se marca como recorrida.

Si durante el proceso se intenta ingresar nuevamente a una posición que ya había sido visitada, el programa detecta que la ruta está repitiendo coordenadas y rechaza el archivo. Esto también permite detectar recorridos que hayan quedado atrapados en un ciclo.

Validación de celdas aledañas:

Antes de avanzar hacia la siguiente posición se calcula la diferencia entre las coordenadas actuales y las siguientes.

Como hipótesis de trabajo se consideró que dos celdas son aledañas cuando comparten uno de sus lados. Por este motivo se permiten solamente movimientos hacia arriba, abajo, izquierda o derecha, y no movimientos diagonales.

Para que un movimiento sea válido, la suma de la diferencia absoluta entre ambas coordenadas debe ser igual a uno.

Por ejemplo, el movimiento de (5,5) a (5,6) es válido, mientras que un movimiento de (5,5) a (5,7) o de (5,5) a (6,6) es rechazado.

Detección de huecos en la ruta:

También se verifica que cada coordenada indicada como siguiente posición tenga realmente un registro asociado dentro del archivo.

Por ejemplo, si una orden ubicada en (10,10) indica que la siguiente posición es (10,11), debe existir una orden cargada en esa coordenada.

Si la posición indicada no existe, se considera que el recorrido posee un "hueco" y el archivo es rechazado. Esta verificación responde al requisito de que la ruta sea completa y permite llegar desde el despegue hasta el final utilizando celdas aledañas.

Detección de órdenes desconectadas

Además de comprobar que sea posible llegar desde el despegue hasta el final, se verifica que todos los registros almacenados en el archivo formen parte de ese mismo recorrido.

Para esto se utilizan dos contadores: cantidadRegistros, que representa la cantidad total de órdenes leídas desde el archivo, y cantidadRecorridas, que indica cuántas posiciones fueron visitadas al seguir la ruta desde el despegue.

Al finalizar el recorrido se comparan ambos valores. Si son diferentes, significa que existen órdenes almacenadas en el archivo que no pueden alcanzarse desde el despegue y, por lo tanto, no forman parte de la misión principal.

En ese caso el archivo también es rechazado.

Carga definitiva en memoria

Solamente después de superar todas las validaciones, el contenido de memoriaTemporal se copia a la matriz principal de memoria.

Al mismo tiempo, la información almacenada en coordenadasUsadas se copia a la matriz ocupada para identificar cuáles son las posiciones que contienen una orden real.

Este procedimiento permite que, si se intenta cargar un archivo inválido, la misión que se encontraba anteriormente en memoria permanezca sin modificaciones.

Con este conjunto de controles se busca asegurar que todo archivo aceptado por el programa representa una única misión completa, consistente y recorrible desde el despegue hasta su finalización.

HIPÓTESIS ADOPTADAS

Durante el desarrollo del trabajo fue necesario establecer algunas hipótesis para resolver situaciones que no se encontraban completamente definidas en la consigna. Estas decisiones se mantuvieron de forma consistente durante la implementación y las pruebas del programa.

- Cada archivo representa una única misión completa. Se considera que una misión comienza con una única instrucción de despegue y finaliza con una instrucción de aterrizaje o con un ataque kamikaze.
- Las celdas aledañas son aquellas que comparten un lado. Por este motivo, los movimientos permitidos son únicamente hacia arriba, abajo, izquierda o derecha. No se consideran válidos los movimientos diagonales.
- Todos los registros de un archivo deben pertenecer al mismo recorrido. No se permiten órdenes almacenadas en coordenadas que no puedan alcanzarse siguiendo la ruta desde el despegue hasta el final.
- El orden físico de los registros dentro del archivo no determina el orden de ejecución. El recorrido se obtiene comenzando desde la instrucción de despegue y siguiendo las coordenadas indicadas por los campos siguientex y siguientey.
- Los campos siguientex y siguientey de una orden final no son utilizados. Cuando se alcanza un aterrizaje o un ataque kamikaze, la operación termina y estos valores dejan de tener efecto. Para los archivos de prueba se adoptó como convención utilizar (0,0) en estos campos.
- Una posición puede contener una orden aunque no posea una acción especial. Una orden puede tener todos los campos de acción en false y representar simplemente un movimiento hacia la siguiente coordenada. Por este motivo se utiliza la matriz auxiliar ocupada para diferenciar una posición realmente utilizada de una posición vacía.
- La validación completa de la misión se realiza al cargar un archivo en memoria. De esta forma, antes de aceptar una misión se verifica que

cumpla las condiciones de despegue, finalización y recorrido establecidas.

Estas hipótesis permitieron definir un criterio único para resolver situaciones ambiguas y mantener un comportamiento consistente en todas las operaciones del programa.

VISUALIZACIÓN HTML

La generación del archivo se realiza desde C++ utilizando las funciones de manejo de archivos de texto. Se utiliza fopen() para crear el archivo, fprintf() para escribir las distintas etiquetas HTML y fclose() para cerrarlo una vez terminada la generación.

Cada celda de la tabla representa una coordenada de la matriz utilizada por el programa. Cuando una posición contiene una orden, se muestra una abreviatura que permite identificar la acción correspondiente. Por ejemplo, D representa un despegue, A un aterrizaje, G1 y G2 el lanzamiento de granadas, K un ataque kamikaze, en una espera y * una posición utilizada únicamente para movimiento.

Para mejorar la visualización se decidió separar los estilos en un archivo CSS independiente. De esta manera, la lógica encargada de generar el contenido HTML permanece separada de la presentación visual.

Debido al tamaño de una matriz de 200 × 200, mostrar todas las posiciones simultáneamente haría que las celdas resultan demasiado pequeñas. Para evitar este problema se creó un área de visualización con desplazamiento horizontal y vertical. La matriz continúa teniendo las 200 × 200 posiciones solicitadas, pero el usuario puede recorrerla mediante las barras de desplazamiento.

También se incorporaron números de filas y columnas para facilitar la identificación de las coordenadas. Los números de las columnas representan el eje Y y los números ubicados a la izquierda representan el eje X. Estos encabezados permanecen visibles mientras el usuario se desplaza por la tabla.

CASOS DE PRUEBA

Para comprobar el funcionamiento del programa se realizaron diferentes pruebas utilizando archivos de ataque válidos e inválidos. El objetivo fue verificar tanto el funcionamiento normal de las distintas operaciones como las validaciones implementadas durante la carga de los archivos.

Se prepararon archivos específicos para provocar cada una de las situaciones que el programa debía detectar. En los casos válidos se verificó que la misión pudiera cargarse correctamente, visualizarse en memoria y representarse mediante el archivo HTML. En los casos inválidos se comprobó que el programa mostrara el mensaje de error correspondiente y evitara cargar el archivo en memoria.

Batería de pruebas:

Caso	Prueba Realizada	Resultado Esperado	Resultado Obtenido
1	Misión válida con 15 registros y aterrizaje	El archivo debe cargarse correctamente	Correcto
2	Misión válida finalizada con ataque kamikaze	El archivo debe cargarse correctamente	Correcto
3	Archivo con dos despegues	Debe ser rechazado	Correcto
4	Archivo sin despegue	Debe ser rechazado	Correcto
5	Despegue acompañado por una acción de ataque	Debe ser rechazado	Correcto
6	Archivo con más de un final	Debe ser rechazado	Correcto

7	Archivo sin aterrizaje ni ataque kamikaze	Debe ser rechazado	Correcto
8	Coordenada repetida	Debe ser rechazado	Correcto
9	Coordenada fuera del rango 0–199	Debe ser rechazado	Correcto
10	Movimiento hacia una celda no aledaña	Debe ser rechazado	Correcto Correcto
11	Hueco en el recorrido	Debe ser rechazado	Correcto
12	Recorrido con un ciclo	Debe ser rechazado	Correcto
13	Registro que no pertenece a la ruta principal	Debe ser rechazado	Correcto
14	Movimiento diagonal	Debe ser rechazado	Correcto

Prueba de una misión válida (Caso 1)

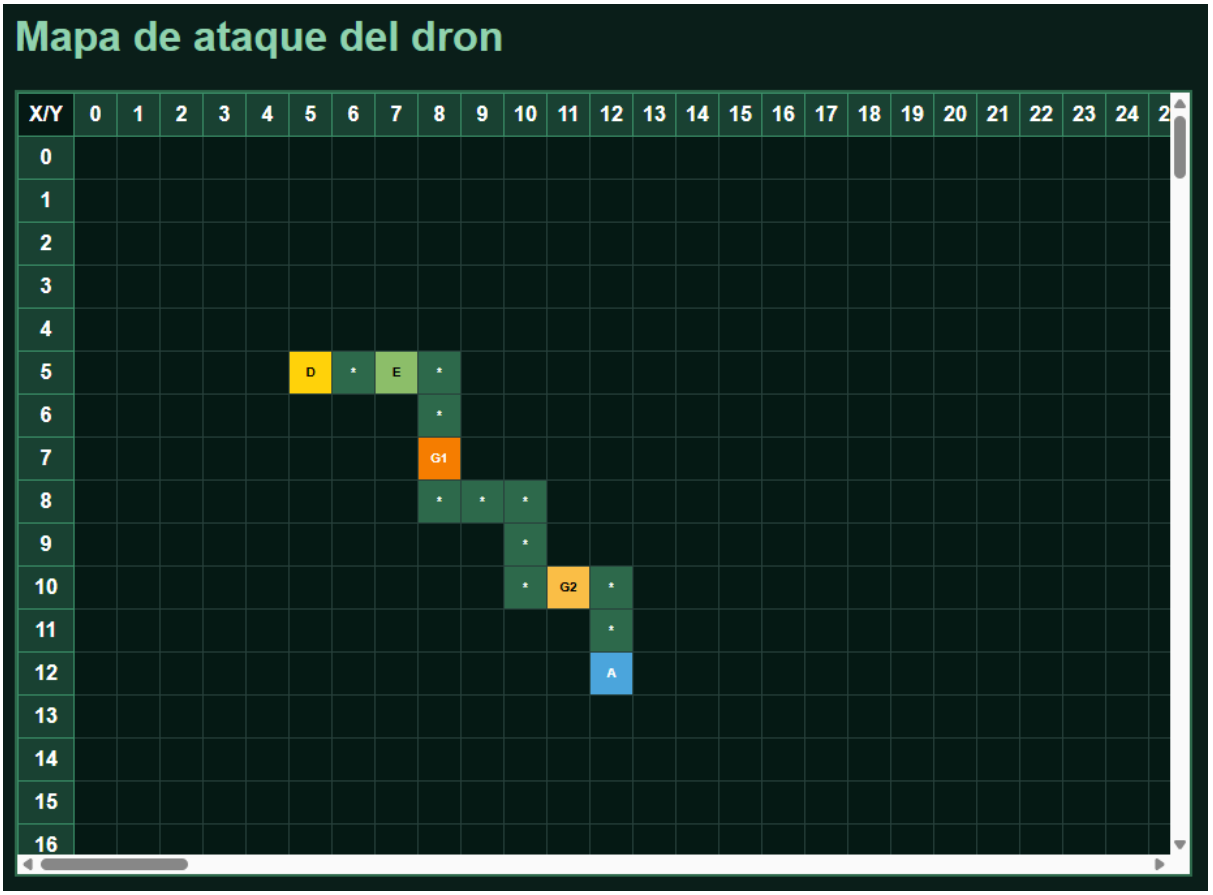
Para verificar el funcionamiento general del programa se utilizó un archivo válido compuesto por 15 registros. La misión incluía una instrucción de despegue, movimientos entre distintas posiciones, tiempos de espera, lanzamiento de granadas y una instrucción final de aterrizaje.

Al cargar el archivo mediante la opción 1, el programa realizó todas las validaciones correspondientes y aceptó la misión correctamente. Luego se utilizó la opción 2 para mostrar las órdenes almacenadas en memoria.

Como la misión contenía más de diez registros, también fue posible comprobar el funcionamiento de la paginación implementada. Después de mostrar los primeros diez registros, el programa esperó que el usuario presionara Enter antes de continuar con las órdenes restantes.

Finalmente, se utilizó la opción 7 para generar la representación HTML de la misión y comprobar que las posiciones cargadas se visualizaran correctamente dentro de la matriz de 200 × 200.

```
Ingrese el nombre del archivo: 01_valido_aterrizaje_15_registros.dat
Archivo cargado en memoria correctamente.
```



Algunas Pruebas de archivos inválidos

Además de probar misiones válidas, se utilizaron distintos archivos preparados específicamente para incumplir alguna de las condiciones de validación. El

objetivo fue comprobar que el programa detectara correctamente cada error y evitará cargar información inválida en memoria.

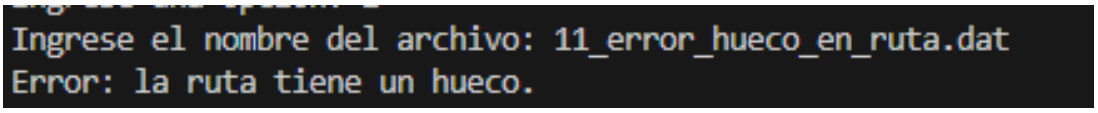
Ruta con un hueco

Se creó un archivo en el cual una orden indicaba como siguiente posición una coordenada para la que no existía ningún registro.

Al intentar cargarlo, el programa detectó que no era posible continuar el recorrido y mostró el mensaje:

Error: la ruta tiene un hueco.

El archivo fue rechazado y no se modificó la memoria principal. Esto permitió comprobar que el programa verifica que todas las posiciones indicadas en el recorrido existan realmente.



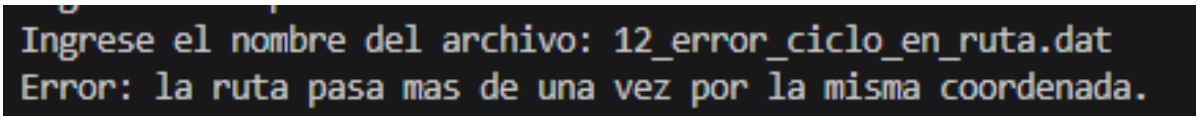
Ruta con un ciclo

También se preparó una misión en la cual el recorrido volvía a una coordenada que ya había sido visitada.

Durante la validación, el programa detectó la repetición y mostró:

Error: la ruta pasa más de una vez por la misma coordenada.

De esta manera se comprobó que la matriz utilizada para registrar las posiciones recorridas permite detectar ciclos y evitar que una misión quede atrapada repitiendo indefinidamente una parte del recorrido.



Registro desconectado del recorrido

En este caso se utilizó un archivo que poseía un recorrido válido desde el despegue hasta el final, pero también contenía una orden adicional que no podía ser alcanzada desde la ruta principal.

El programa detectó esta situación mediante la comparación entre la cantidad total de registros del archivo y la cantidad de posiciones efectivamente recorridas.

El resultado obtenido fue:

Error: existen órdenes que no forman parte de la ruta.

Esto permitió verificar que no alcanza con que exista un recorrido correcto hasta el final, sino que todas las órdenes almacenadas deben formar parte de esa misma misión.

```
Ingrese el nombre del archivo: 13_error_registro_desconectado.dat
Error: existen ordenes que no forman parte de la ruta.
```

UTILIZACIÓN DE LA IA

Durante el desarrollo del trabajo práctico se utilizó ChatGPT como herramienta de apoyo en distintas etapas del proyecto.

La principal colaboración de la herramienta estuvo relacionada con el diseño y la organización de la lógica de validación de los archivos de ataque. Esta parte resultó ser una de las más complejas del trabajo, debido a la cantidad de condiciones que debían verificarse de manera conjunta antes de aceptar una ruta como válida.

ChatGPT fue utilizado para analizar cómo dividir estas validaciones, en qué orden realizarlas y qué estructuras auxiliares podían utilizarse para resolverlas de una manera clara.

Primero leería todos los registros del archivo y los cargaría en una matriz temporal, no directamente en la memoria principal. Mientras hacés esa lectura podés controlar varias condiciones simples:

- verificar que x e y estén entre 0 y 199;
- usar una matriz bool coordenadasUsadas[200][200] para detectar coordenadas repetidas;
- contar cuántos despegues hay;
- guardar la posición del despegue;
- contar aterrizajes y ataques kamikaze;
- verificar que el despegue no tenga una acción de ataque.

Una vez terminada la lectura, si esas condiciones básicas son correctas, recién ahí recorrés la misión.

Empezás desde las coordenadas del despegue y seguís:

siguientex
siguientey

Para saber si ya pasaste por una posición podés usar otra matriz:

bool recorrido[200][200];

Cada vez que llegás a una posición la marcás en true. Si volvés a encontrarla en true, significa que la ruta repite una coordenada o entró en un ciclo.

Antes de pasar a la siguiente posición también deberías comprobar:

1. que esté dentro de la matriz;
2. que sea una celda aledaña;
3. que realmente exista un registro en esa coordenada.

Al final podés contar cuántos registros había en el archivo y cuántas posiciones recorriste. Si los valores son distintos, significa que existen órdenes que no forman parte de la ruta principal.

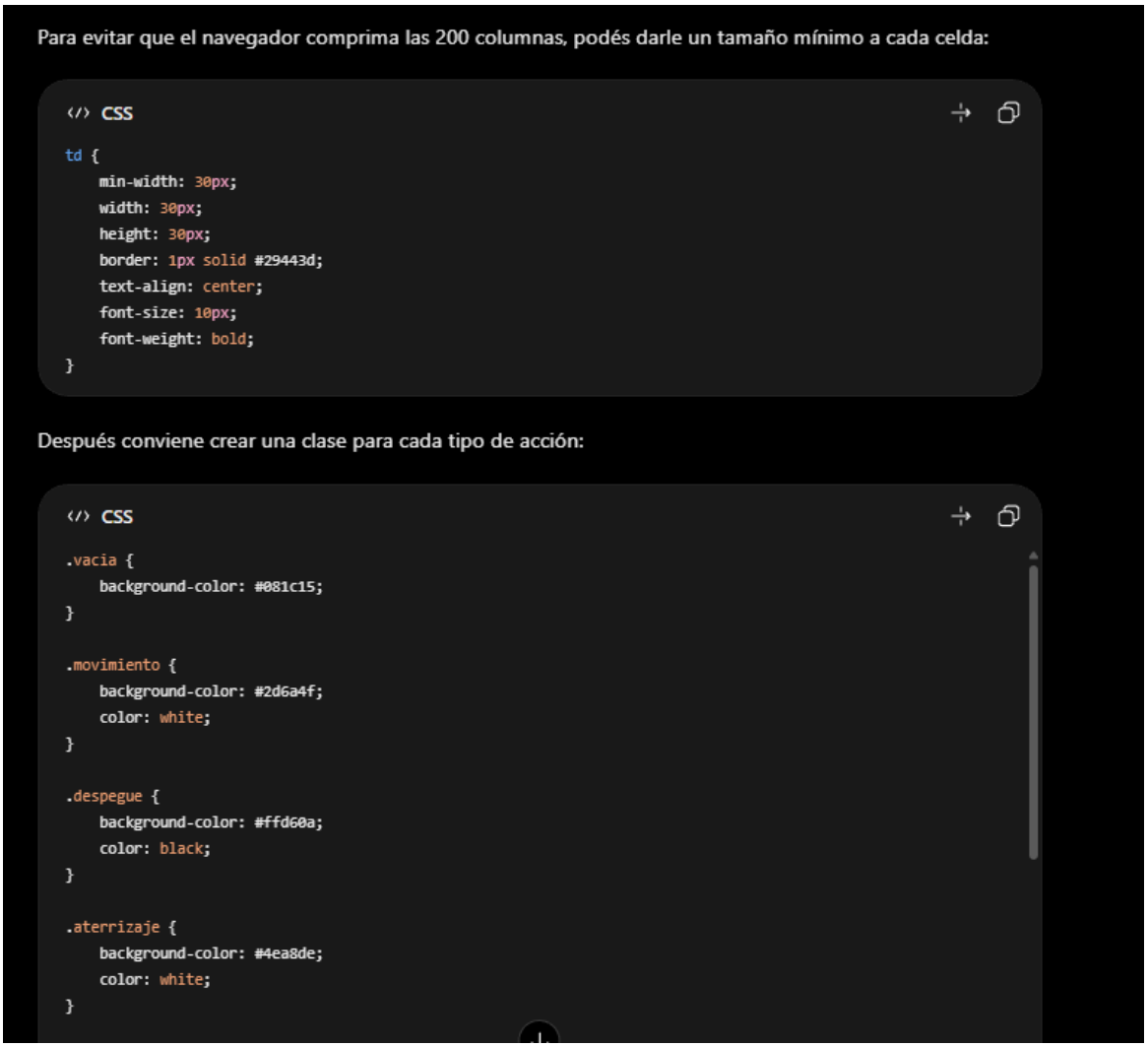
Solamente si todas estas validaciones pasan correctamente copiaría la matriz temporal a la memoria principal."

Se siguió esta organización utilizando matrices booleanas, contadores y ciclos simples. Se descartaron alternativas con estructuras más avanzadas para mantener la implementación dentro de los contenidos trabajados en la materia.

Otra colaboración importante fue la generación de una batería de casos de prueba. Se utilizaron archivos preparados para representar tanto misiones válidas como distintas situaciones incorrectas. Esto permitió comprobar individualmente cada una de las validaciones implementadas y verificar que el programa rechazara los archivos incorrectos sin modificar la información previamente cargada en memoria.

También se utilizó ChatGPT como apoyo para la parte de visualización en HTML y CSS. Se contaba previamente con conocimientos básicos de

maquetado web utilizando HTML y CSS, por lo que la herramienta fue utilizada principalmente para facilitar el armado de algunos estilos y mejorar la presentación de la matriz de 200 × 200. A partir de esto se incorporaron elementos como el desplazamiento horizontal y vertical, los encabezados de coordenadas y diferentes estilos para representar las acciones del recorrido.



Las respuestas proporcionadas por ChatGPT fueron utilizadas como orientación y no incorporadas de forma automática. Las soluciones fueron revisadas, adaptadas al código del proyecto, simplificadas cuando fue necesario y posteriormente verificadas mediante pruebas realizadas sobre el programa.

CONCLUSIÓN

El desarrollo de este trabajo permitió comprender que la resolución de un problema no depende únicamente de lograr que el programa funcione, sino también de poder organizar correctamente la lógica, anticipar situaciones incorrectas y tomar decisiones claras frente a casos que no se encuentran completamente definidos.

Uno de los principales desafíos fue transformar una consigna general en una serie de reglas y condiciones que pudieran ser verificadas de manera ordenada. Esto requirió analizar distintas alternativas, plantear hipótesis y realizar pruebas que permitieran comprobar si las decisiones tomadas eran correctas.

El proceso de prueba también resultó fundamental, ya que permitió detectar situaciones que inicialmente no eran evidentes y comprobar el comportamiento del programa frente a distintos escenarios. De esta manera, el desarrollo no se limitó a implementar una solución, sino que implicó revisar, corregir y justificar las decisiones adoptadas.

En conclusión, el trabajo permitió integrar los conocimientos vistos durante la materia en un problema de mayor complejidad y, principalmente, desarrollar una forma más estructurada de analizar, diseñar y validar una solución antes de considerarla terminada.